

CreditPassport

Credit history you can prove, not just claim. Payments that provably happened on another chain become a credit profile on Creditcoin, underwritten by an agent whose discretion is bounded on-chain. No oracle operator anywhere.

Verified history

Every record is an InvoicePaid log decoded from a receipt whose inclusion Creditcoin's verifier precompile checked.

Bounded agent

The agent scores and explains. The contract computes the maximum credit limit from verified data and refuses anything above it.

Drawable credit

Payers draw cUSD against the limit on Creditcoin and repay. Limits rise only with more proven history.

PROBLEM

Credit needs history. History lives where payments happen.

Moving a payment record from one chain to a lender on another requires either a centralized oracle that asserts "this address paid its invoices," or self-reported data. Both put a trusted party between the facts and the lender, and both are how credit data gets faked.

The same gap shows up everywhere people ask for portable reputation:

"LinkedIn but for landlords and tenants" — two-way verified rental history, r/SomebodyMakeThis

"Checks if the software you paid for actually works" — pre-payment milestone proof for freelancers, r/SideProject

"Referral agreements with automated commission escrow" — verbal promises lose money, r/SomebodyMakeThis

"A CFA-like credential that verifies real code" — Ask HN

Attestcoin turns a source-chain receipt into something a Creditcoin contract can check itself.

- Creditcoin attests Sepolia blocks continuously. On testnet the attestation trails the head by ~36 blocks, about 7 minutes.
- The hosted prover returns Merkle inclusion + continuity proofs and the transaction bytes, including the receipt.
- The verifier precompile at 0xFD2 checks the proof on-chain. EvmV1Decoder decodes the receipt's logs.
- So a contract can enforce: "this payer paid this payee this amount by this block" without anyone asserting it.

What that unlocks

A credit profile assembled only from proven payments, an underwriting agent that can read it but not inflate it, and a credit line whose ceiling is a pure function of verified data. The trust roots are Creditcoin's attestation and the owner's registration of which source contracts count. Nothing else.

Five steps, two chains, zero oracles

1 Pay

Payer settles an invoice through PaymentRail on Sepolia. The log carries amount, due block and paid block.

2 Attest

Creditcoin attests the Sepolia block a few minutes later.

3 Prove

The agent fetches inclusion and continuity proofs, singly or ten at a time sharing one continuity proof.

4 Verify

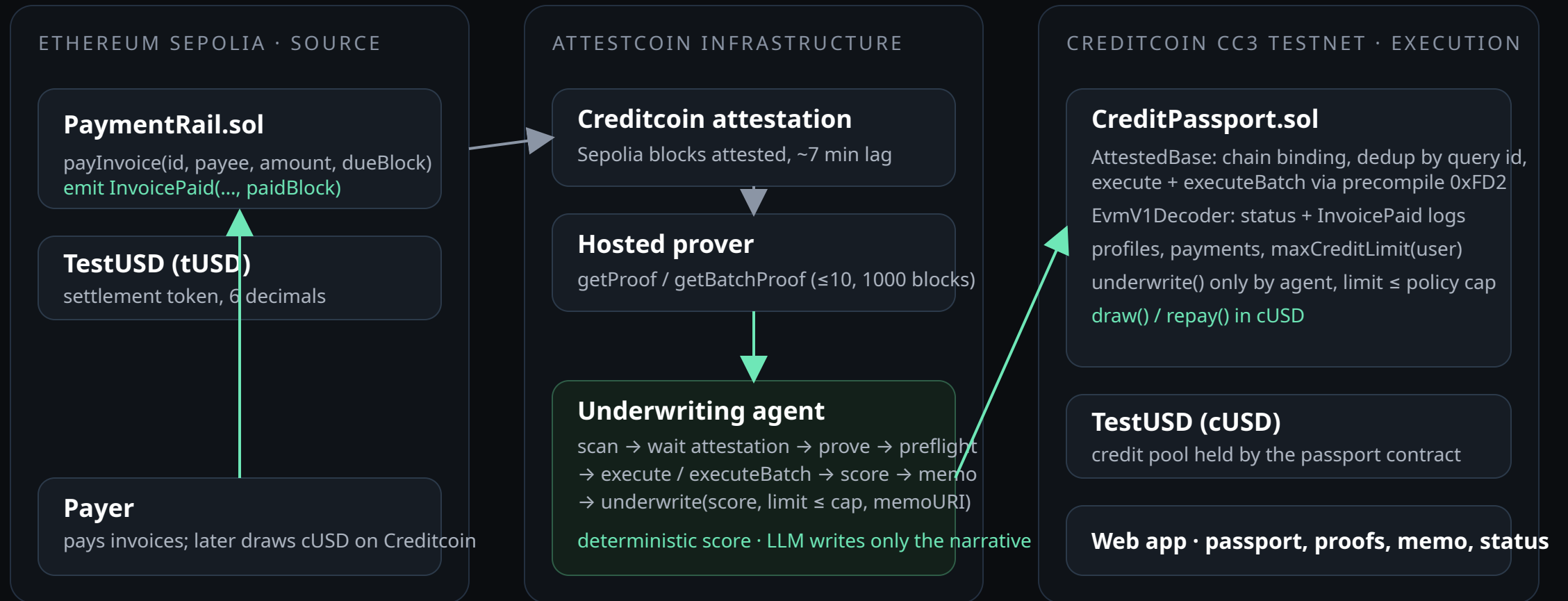
CreditPassport asks the precompile to verify, decodes the receipt, keeps only rail logs, records on-time or late.

5 Underwrite

The agent scores the verified history, writes a memo, sets a limit the contract caps. The payer draws cUSD.

Lateness is decided by `dueBlock` and `paidBlock` fixed in the source log at payment time, never by whoever submits the proof.

Two chains, one prover, one agent



Every layer of the protocol is exercised, and tested against real prover output

WHERE	ATTESTCOIN SURFACE	WHY IT MATTERS
<code>AttestedBase.execute</code>	<code>INativeQueryVerifier.verifyAndEmit</code> (single), <code>calculateTxIndex</code>	Inclusion + continuity verified on-chain; query id derivation identical to ASCBase for replay protection
<code>AttestedBase.executeBatch</code>	<code>verifyAndEmit</code> batch overload, shared continuity proof	Up to 10 payments imported in one transaction; not in the tutorials
<code>CreditPassport._processAndEmitEvent</code>	<code>EvmV1Decoder.decodeReceiptFields</code> , <code>getLogsByEventSignature</code>	Receipt status checked; only logs from the registered rail count
<code>agent/src/proofs.ts</code>	<code>@gluwa/usc-sdk ProofBuilder.getProof / getBatchProof</code> , <code>waitUntilHeightAttested</code>	Waits for attestation, fetches proofs, flattens batches into contract arguments
<code>agent/src/proofs.ts</code>	<code>PrecompileBlockProver.verifySingle</code> , <code>PrecompileChainInfoProvider</code>	Off-chain dry run before spending gas; live attestation lag in the UI
<code>test/RealProverBytes.t.sol</code>	Captured prover txBytes for a live Sepolia transaction	Decoder reproduces the Sepolia receipt field-for-field; <code>execute()</code> rejects it only for lacking a rail log
<code>npm run cli -- verify</code> · fetches a proof for any Sepolia tx and asks the precompile, no key, no gas		
<code>checks/LivePrecompileCheck.sol</code>	Deploy + <code>setSources</code> + execute inside one	Real Sepolia ERC-20 transfer recorded on a

An agent you do not have to trust

The agent reads only what the contract verified, computes a deterministic score (0–1000) from punctuality, depth, tenure, volume and recency, and writes an underwriting memo. When a model key is configured, a language model writes the two-sentence narrative from the computed facts. It never produces a number.

Then it calls `underwrite(user, score, limit, memoURI)`. The contract computes the ceiling itself and reverts above it.

```
datedCap = 50% × datedVolume × onTime /  
(onTime + late)  
undatedCap = 25% × undatedVolume  
if late > onTime then datedCap = 0  
maxCreditLimit = datedCap + undatedCap  
  
require(limit ≤ maxCreditLimit && limit ≥  
drawn)
```

Demo: Alice, 6 proven invoices, 5 on time → score 814, cap 606.25 cUSD, agent extends 606. Bob, 1 on time, 2 late → score 477, cap 0, agent extends 0.

What an attacker would have to do

- **Fake a payment log.** Logs are accepted only from the registered rail; a look-alike contract emitting InvoicePaid is ignored, and a transaction with no rail log reverts.
- **Replay a proof.** Query id (chainKey, block, txIndex) is marked before the hook runs; batches skip processed entries and revert only if nothing is new.
- **Use another chain.** Proofs must carry the chain key fixed at deployment.
- **Prove a failed transaction.** Receipt status is checked; reverted source transactions record nothing.
- **Backdate a payment.** Due and paid blocks come from the source log fixed at payment time.
- **Bribe the agent.** The agent's maximum is a pure function of verified history. A compromised agent can under-extend, never over-extend, and cannot lower a limit below what is drawn.

Trust roots: Creditcoin's attestation of Sepolia, and the owner's registration of the rail and token addresses. There is no oracle operator to compromise.

DEMO

A passport, end to end

CreditPassport

How it worksAttestcoin docs

local anvil · block 1,539agent offline

PASSPORT

0x70997970C51812dc3A010C7d01b50e0d17dc79C8

Payer address, e.g. 0x7099...79C8

Q Open passport

Score

814

/ 1000 · Strong

Verified payments

6

On time

5 of 6 (83%)

Late

1

Dated volume

1,455 tUSD

First verified payment

block 11,600,040

Most recent

block 11,622,050

Credit limit

606 cUSD

Policy cap 606.25 cUSD · agent extended 100% of it

Available to draw

403.92 cUSD

Drawn 202.08 cUSD · 33% utilised

Underwriting memo

Score 814/1000 from 6 verified payments (5/6 dated invoices on time, 0 undated transfers, 1455.00 settled). Policy cap 606.25; extending 606.00 (100% of cap). Every figure derives from Attestcoin-proven source-chain receipts.

templateWed, 02 Sep 2026 23:34:39 GMT · recorded at Creditcoin block 41

Factor	Points	Detail
base	+300	Starting point for any wallet with at least one verified payment.
payment-behaviour	+293	5 of 6 dated invoices paid on time (83%). 1 late.
track-record	+150	6 dated and 0 undated verified payments.
tenure	+31	Verified history spans 22010 source blocks (~3.1 days).
volume	+40	Verified settlement volume 1455.00 (undated transfers at half weight).
recency	0	Last verified payment 0.1 days ago.

Verified payments

Invoice	Payee	Amount	Due block	Paid block	Result	Source tx	Verified on Creditcoin
0xb47deb6a...	0x0000...cafe	90	11,621,900	11,622,050	late by 150 blocks	#11,622,050/5	0xa1fa60aa...c027c8
0x7a4f33bf...	0x0000...cafe	205	11,621,800	11,621,600	on time	#11,621,600/4	0xdd55b41b...15b381
0x9395b9bb...	0x0000...cafe	310	11,621,200	11,621,000	on time	#11,621,000/3	0xdd55b41b...15b381
0xad502b5f...	0x0000...cafe	120	11,614,500	11,614,400	on time	#11,614,400/2	0x2673f563...118459
0x8410fc32...	0x0000...cafe	480	11,607,300	11,607,250	on time	#11,607,250/1	0x2bf268a8...3ac62d
0x7f937f62...	0x0000...cafe	250	11,600,100	11,600,040	on time	#11,600,040/0	0x3a3bd7d2...173334

Query id for each row is keccak256(chainKey, sourceBlock, txIndex); the same identity the contract uses to refuse replays.

Score, credit limit and available credit at the top; the agent's memo with its factor table; every verified payment with the source block and its Creditcoin verification transaction; the on-chain cap breakdown; and the underwriting history.

Two rows share one Creditcoin transaction: they were imported together through executeBatch.

```
$ npm run cli -- verify
tx 0x197cff58...68ec0
block 11622748 (latest attested 11622750)
proof fetched in 82 ms: 8 Merkle siblings, 3 continuity
roots
verifier precompile 0xFD2 on Creditcoin says VALID
```

What exists today

Contracts

AttestedBase, CreditPassport, PaymentRail, TestUSD. 32 tests including genuine prover bytes. Deploy scripts for both chains.

Agent

Single and batch proof submission with off-chain preflight, deterministic scoring, memos as data: URIs, status API, CLI for pay / prove / underwrite / profile / verify.

Web

Passport dashboard with live attestation lag and agent queue. Empty, loading and error states.

Deployed on CC3 testnet

CreditPassport: see README · PaymentRail (Sepolia): see README

Roadmap

- Ethereum mainnet as source (chain key 3) and more rails: ERC-3009 transfers, payroll, subscriptions
- Creditcoin-native lenders consuming passports as collateral signals
- Attestcoin write-ability to push approved lines back to the source chain
- Scanned-statement ingestion via the same verify-not-trust pattern